

Getränkebestellungen am Rattenfest

Studienarbeit
Herbstsemester 2026



Version: Lokaler Build
Datum: 01.01.1980

Team: Andrin Klarer
Jasmin Fässler
Betreuer: Frieder Loch

Departement Informatik
Ostschweizer Fachhochschule



Inhaltsverzeichnis

1	Einleitung	6
1.1	Ausgangslage	6
1.2	Systemkontext und Domäne	6
1.3	Ziel der Arbeit	6
1.4	Rahmenbedingungen	6
1.5	Vorgehen	6
1.6	Problem Domain	8
2	Vorarbeiten	9
2.1	Prototyp 2026	9
2.1.1	Funktionsumfang	9
2.1.2	Erfahrungen aus dem Einsatz	10
2.1.3	Grenzen	10
2.2	Abgrenzung zu dieser Arbeit	10
2.3	Risikoanalyse	10
2.3.1	Technische, allgemeine Risiken	10
2.3.2	Spezielle Risiken	11
2.4	Bisherige Lösung mit SumUp	11
3	Was können wir von vorhandenen Lösungen abschauen	12
3.1	Verwandte Lösungen und Technologien	12
3.1.1	Können wir Technologien übernehmen von ähnlichen Bereichen?	12
3.1.2	Webshop für Getränke	12
3.1.3	Bestell-App für Zuschauer im Fussballstadion	13
3.1.4	Fazit	14
3.2	Nutzungskontext am Fest	15
3.2.1	Restaurant und Bars Umgebung	15
3.2.2	Accessibility Guidelines übertragen	15
3.2.3	Guidelines für Rattenfest Website	16
3.2.4	Dark Mode	18
3.3	Erkenntnisse	18
4	Anforderungen	19
4.1	Akteure	19
4.2	User Stories und Functional Requirements	19
4.2.1	Vorbereitung	19
4.2.2	Bestellung während des Fests	19
4.2.3	Bearbeitung und Abholung	20
4.2.4	Rücknahme und Abrechnung	20
4.3	Priorisierung	21
4.4	Non-Functional Requirements	23
5	Design und Architektur	25
5.1	Technologieentscheid	25
5.1.1	Kriterien des Frameworks	25
5.1.2	Vorauswahl	25
5.1.3	Informationen	27
5.1.4	React / Next.js mit Typescript	27
5.1.5	Angular	27
5.1.6	Vue / Vue.js	27
5.1.7	ASP.NET	28

5.1.8	Fazit	28
5.2	Architektur	28
5.3	Datenmodell	28
5.4	UI-Konzept	28
5.5	Zugangskonzept	28
6	Implementation	29
7	Ergebnisse	30
7.1	Erfüllung der Functional Requirements	30
7.2	Erfüllung der Non-Functional Requirements	30
7.3	Usability-Tests	30
8	Schlussfolgerungen und Ausblick	31
8.1	Diskussion	31
8.2	Ausblick	31
9	Projektmanagement	32
9.1	Vorgehen	32
9.2	Zeitplan und Meilensteine	33
10	Glossar	34
11	Hilfsmittelverzeichnis	35
12	Bilderverzeichnis	35
13	Tabellenverzeichnis	36
14	Bibliografie	36
15	Anhang A	38
15.1	APIs	38
15.2	Screenshots	38
15.3	Testreports	38
16	Anhang B	38
16.1	Eigenständigkeitserklärung	38

Abstract

Management Summary

Vorgehen und Technologien

Ergebnisse

1 Einleitung

1.1 Ausgangslage

Das Rattenfest ist ein jährlich stattfindendes Festival, das von Studierenden der OST organisiert wird und rund 3'000 Personen besucht. Die Getränke werden an mehreren Bars ausgeschenkt, die unabhängig vom Rattenfest betrieben werden. Damit nicht jede Bar ihre Versorgung selbst organisieren muss, betreibt das Rattenfest ein zentrales Getränkelager, aus dem die Bars während des Fests laufend beziehen.

Dieser Prozess wurde in den letzten Jahren versucht zunehmend zu digitalisieren, die bisherigen Lösungen blieben jedoch unzureichend.

1.2 Systemkontext und Domäne

Es gibt eine Website des Rattenfest `rattenfest.ch`, diese wird über Hostpoint gehostet. Das Ticket-system und die Bezahlung läuft extern über einen Anbieter. Die Kommunikation intern läuft über Microsoft Teams, E-Mails und Chatdienste wie WhatsApp. Dateien sind unter Microsoft Teams abgelegt, es gibt keinen Dateiserver. Es gibt bereits einen Prototyp für das Barsystem und als Hilfe für den OK im Vorverkauf wurde bereits eine kleine Website getestet. So sind die Schnittstellen vom Rattenfest sehr flexibel und die verschiedenen Services können unabhängig voneinander gewechselt werden.

1.3 Ziel der Arbeit

Ziel dieser Arbeit ist eine Webapplikation, die den gesamten Lebenszyklus der Getränke abbildet, von der ersten Bedarfsschätzung bis zur Abrechnung nach dem Fest:

- **Vor dem Fest** erfassen die Bars ihre Bedarfsschätzung in der Applikation. Das OK prüft und korrigiert diese und leitet daraus die Bestellung beim Getränkehändler ab. Die gelieferte Ware bildet den Anfangsbestand des Lagers.
- **Während des Fests** bestellen die Bars laufend aus diesem Bestand. Das Lagerteam stellt die Bestellungen bereit und informiert die Bars über den Status.
- **Nach dem Fest** lässt sich für jede Bar eine detaillierte Abrechnung erstellen.

Neben der Funktionalität soll die Lösung auf das Rattenfest zugeschnitten und für den langfristigen Einsatz geeignet sein. Dabei sind drei Eigenschaften besonders wichtig:

- **Bedienbarkeit** unter den Bedingungen am Fest: dunkel, laut, unter Zeitdruck und mit Personal, dessen Aufmerksamkeit im Verlauf des Abends nachlässt.
- **Zuverlässigkeit** während des Fests. Sobald die Bars auf den manuellen Weg ausweichen müssen, verliert die Applikation ihren Zweck.
- **Verwendbarkeit und Wartbarkeit** durch ein jährlich wechselndes OK mit unterschiedlichen technischen Kenntnissen.

1.4 Rahmenbedingungen

Die Arbeit wird als Studienarbeit (SA) im Umfang von 8 ECTS durchgeführt, was einem Aufwand von rund 240 Stunden entspricht.

1.5 Vorgehen

Um diese Ziele zu erreichen, machen wir diese Schritte:

1. Problem genauer definieren

2. Wie wurde es bisher gelöst
3. Recherche
 1. Vorarbeit mit Prototyp wird analysiert und Erfahrungen für Anforderungen gesammelt >
 2. Risikoanalyse
 3. Konkurrenzanalyse: was gibt es bereits, was können wir abschauen
 1. Technologien
 2. Webshops
 3. ähnliche Umgebung Fussballstadion
 4. Benutzung in spezieller Umgebung verbessern > betrachte ähnliche Beeinträchtigungen, erstelle eigene Guideline wie WCAG mit Elementen auch von WCAG
4. alles gelernte aus der Recherche in Anforderungen übernehmen, deklarieren woher
5. Umsetzung der Website
 1. Technologie auswählen
 2. DB Domain Modell und Architektur planen wo nötig
 3. Programmierung
 4. kleiner User Test an simulierter Umgebung
 5. Deployment
 6. Bedienungsanleitung für RF OK und BAR MA

1.6 Problem Domain

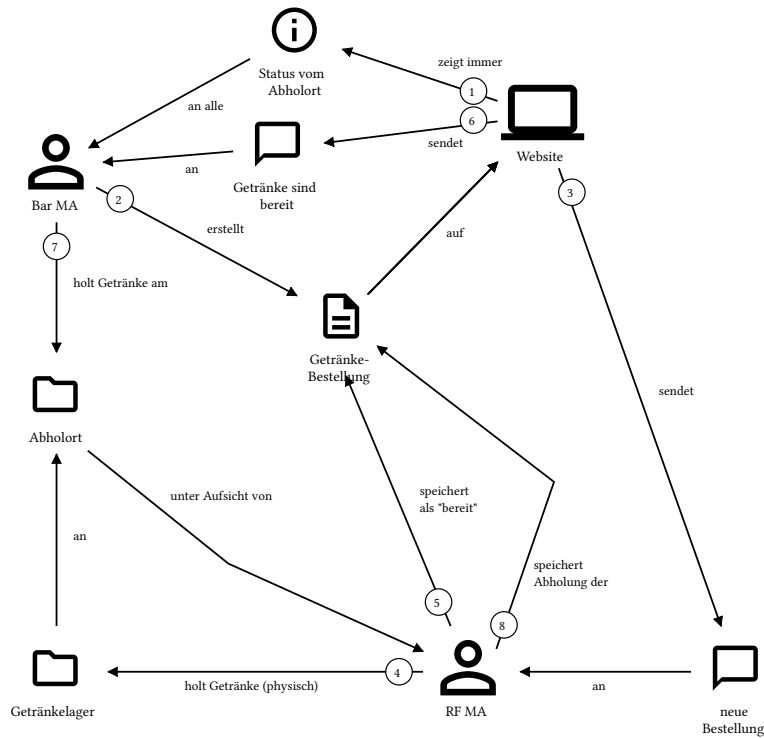


Abbildung 1: Problem Domain der Getränkebestellung

Umfang:

- Eine Getränke-Bestellung umfasst wenige bis zu mehreren hundert Artikel.
- Das Bereitstellen der Getränke (4) dauert je nach Grösse der Bestellung und Anzahl der eingehenden Bestellungen zwischen 2 und 10 Minuten.
- Gemäss den Erfahrungen aus dem letzten Jahr gehen zu Spitzenzeiten bis zu 10-15 Bestellungen pro 15 Minuten ein.
- Die Wege bei der Abholung (7) sind unterschiedlich lang. Bis man sich mit der Ware durch die Menschenmenge gekämpft hat, können mehrere Minuten vergehen.

Höchste Priorität bei der Bestellung: Solange ein Produkt an Lager ist, darf es an keiner Bar ausgehen. Das wird durch schnelle Kommunikation und Live-Updates sichergestellt.

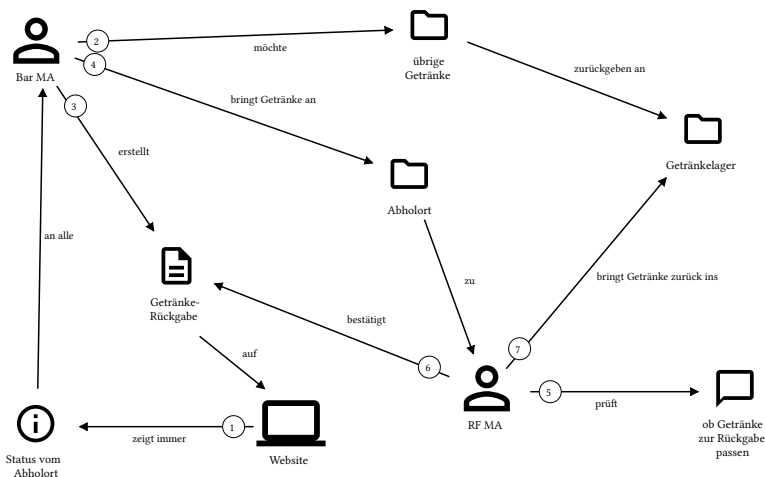


Abbildung 2: Problem Domain der Getränkerückgabe am Ende des Festes

Bei der Rückgabe ist das Fest vorbei und es darf deshalb auch länger dauern.

2 Vorarbeiten

In den vergangenen Jahren wurde versucht, das Getränkemanagement am Rattenfest zunehmend zu digitalisieren. Für das Rattenfest 2026 wurde, unabhängig von dieser Arbeit und vor deren Beginn, erstmals ein Prototyp für die Getränkebestellungen während des Fests entwickelt und eingesetzt. Dieser Prototyp wird im Folgenden beschrieben und von den Ergebnissen dieser Arbeit abgegrenzt.

2.1 Prototyp 2026

Funktionsumfang

Der Prototyp deckt ausschliesslich den Bestellprozess während des Fests ab:

- Die Bars bestellen über eine Webseite Getränke aus dem Lager.
- Das Lagerteam erhält die Bestellungen, bearbeitet diese und stellt sie bereit.
- Die Bars holen die bereitgestellten Getränke im Lager ab.
- Am Ende des Fests können Retouren erfasst werden.
- Nach dem Fest stehen die Daten zur Verfügung, um pro Bar eine Abrechnung zu erstellen.

Für die Benutzeroberfläche wurde die Komponentenbibliothek shadcn/ui [1] verwendet, die unter anderem einen Dark Mode mitbringt. Die Applikation übernahm dabei die Einstellung des Geräts und wurde dunkel dargestellt, sofern dieses auf den Nachtmodus geschaltet war.

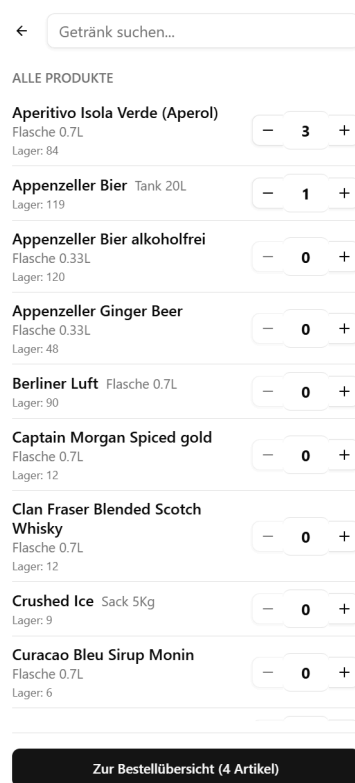


Abbildung 3: Bestellung durch eine Bar

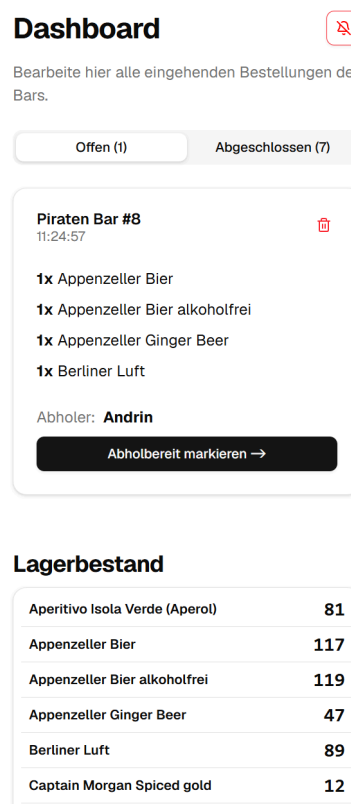


Abbildung 4: Übersicht im Lager

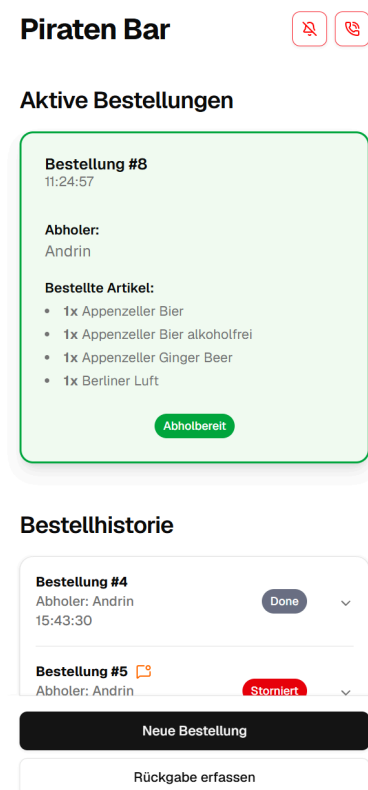


Abbildung 5: Ansicht der Bar

Erfahrungen aus dem Einsatz

Der Prototyp war während des gesamten Fests ohne nennenswerte Ausfälle im Einsatz. Die Rückmeldungen von Bar- und Lagerteam fielen überwiegend positiv aus. Der Einsatz unter realen Bedingungen lieferte zudem konkrete Hinweise auf fehlende Funktionen, die ohne Prototyp kaum erkannt worden wären, beispielsweise:

- die Berücksichtigung von Pausen und Schichtwechseln,
- Angabe von Preisen und Übersicht der Ausgaben,
- Einfache Kontaktmöglichkeit zur Security.

Diese Erkenntnisse fließen als Anforderungen in diese Arbeit ein.

Grenzen

Trotz des erfolgreichen Einsatzes genügt der Prototyp den Zielen dieser Arbeit nicht:

- **Architektur und Qualität:** Der Prototyp wurde aufgrund mangelnder Planung als Einweglösung entwickelt. Es besteht keine Testabdeckung und keine saubere Architektur, was eine Weiterentwicklung und Wartung durch ein wechselndes OK erschwert.
- **Datenmodell:** Das Datenschema ist auf ein einzelnes Fest ausgelegt. Um die App für ein weiteres Fest zu verwenden, muss die DB gewiped (gelöscht) werden und sämtliche Daten gehen verloren, womit ein Vergleich zwischen Jahrgängen nicht möglich ist.
- **Funktionsumfang:** Abgedeckt ist nur der Bestell- und Rücknahmeprozess. Bedarfsschätzung und Vorbestellung, automatischer Lagerbestand, Preisaufstellungen und fertige Abrechnungen fehlen vollständig.
- **Administration:** Viele Aufgaben wie der Import von Getränken, Bars, Benutzern und Telefonnummern erfolgen über das Dashboard der Datenbank oder mit Skripten. Ohne technische Kenntnisse ist die Applikation nicht betreibbar.

2.2 Abgrenzung zu dieser Arbeit

Der Prototyp dient dieser Arbeit als validierte Ausgangslage, nicht als Codebasis, die unverändert übernommen wird. Da der Umfang dieser Arbeit deutlich grösser ist, erlauben wir uns bewiesene Konzepte zu übernehmen.

- der Technologie-Stack, da er sich im Einsatz bewährt hat und für ein wechselndes OK einfach zu warten ist. (Vercel & Supabase)
- der grundlegende Ablauf des Bestellprozesses & Retouren
- das Zugangskonzept: Bars greifen über einen persönlichen Link ohne Passwort zu, Lagerteam und Administratoren melden sich mit Benutzername und Passwort an.

Neu im Rahmen dieser Arbeit entstehen insbesondere ein über mehrere Jahre hinweg nutzbares Datenmodell und eine testbare Architektur. Ausserdem wird die Administration ergänzt, damit gibt es eine Verwaltung der Applikation selbst sowie der Prozesse vor und nach dem Fest. Zudem wird die Bedienbarkeit unter Festbedingungen, auf Basis der Recherche und den Rückmeldungen aus dem Einsatz, untersucht und weiterentwickelt.

2.3 Risikoanalyse

Technische, allgemeine Risiken

Sicherheit Code .. Ausfall

Spezielle Risikos

Risiko eine Funktion zu vergessen Massnahme: Zielgruppe befragen, Im Voraus mit Prototyp und im Nachhinein User Tests machen

Risiko, dass es auf einer Plattform/Betriebssystem/Bildschirmgrösse nicht funktioniert wie gedacht
Massnahme: tests

Risiko, dass sich die Benutzung durch die spezielle Umgebung verschlechtert. Spezielle Umgebung: während Rattenfest

- ausserhalb von der Website: Tipps für Bars, wie sie das Gerät mit der Website einrichten können, damit es möglichst integriert und nicht lästig ist (Gerät, Halterung, Rolle zuweisen ?)
- Bedienung von Website (User Experience) eng, dunkel, laut, Bedienung eingeschränkt

Massnahme: Recherche, stärkerer Fokus auf Accessibility

Risiko, die Internetverbindung ist zu langsam, da es sehr viele Menschen hat
Massnahme: Tests
> ist das Problem auch mit anderen Diensten? Protokolle analysieren, was lässt sich verbessern?
Nutzwertanalyse für teureres, lokales Hosting oder günstiges Hosting mit weniger Setupaufwand?
Möglichkeiten für Hosting am Rattenfest mit einfachem Setup? Bluetooth? Etwas wie Walkie Talkie?
Seiteninhalt klein halten und auf Smartphones cachen. Inhalt anzeigen + funktionen, auch wenn es noch nicht alles geladen hat.

Risiko, dass die nachfolgenden RF-OK Personen die Website nicht bedienen können und es nach z. B. zwei OK-Wechsel nicht mehr benutzt wird.
Massnahme: kurze, einfach verständliche, einfach auffindbare und erreichbare Anleitungen erstellen. Kontakt vermerken für Fragen. Wissen gut weitergeben.

2.4 Bisherige Lösung mit SumUp

Bis zum Rattenfest 2026 wurde die Getränkeausgabe mit dem Kassensystem von SumUp erfasst. Dazu war auf der SumUp-Kasse pro Bar ein Tisch eingerichtet. Bar-Mitarbeitende kamen zum Lager und gaben ihre Bestellung mündlich auf, das Lagerteam erfasste sie in der Kasse und holte die Getränke aus dem Kühlwagen. Nach dem Fest wurden die Daten pro Tisch exportiert und für die Abrechnung verwendet.

Das System erfüllte seinen Zweck nur teilweise, da es für den Verkauf und nicht für die Lagerverwaltung gedacht ist:

- **Wege und Wartezeiten:** Bar-Mitarbeitende mussten für jede Bestellung zum Lager gehen und dort warten, bis sie bereitstand. In dieser Zeit fehlten sie hinter der Bar.
- **Mehraufwand im Lager:** Das Lagerteam musste neben dem Bereitstellen jede Bestellung zusätzlich erfassen.
- **Kein Lagerbestand:** Der aktuelle Bestand war im System nicht ersichtlich.
- **Gebinde und Rückgaben:** Ausserdem kam es zu Unklarheiten bei Gebindegrössen, Rückgaben liessen sich nur umständlich erfassen. Fehler konnten nicht korrigiert werden und mussten in der Abrechnung berücksichtigt werden.

3 Was können wir von vorhandenen Lösungen abschauen

3.1 Verwandte Lösungen und Technologien

Zuerst betrachten wir Technologien von ähnlichen Bereichen, danach schauen wir was das UI von einem Webshop abschauen kann und was anders sein soll.

Können wir Technologien übernehmen von ähnlichen Bereichen?

Neben Bestellsystemen haben wir stichprobenweise Technologien aus verwandten Bereichen betrachtet. Leitfrage war jeweils, ob sie einen Arbeitsschritt am Rattenfest (Bestellen, Bereitstellen, Abholen, Rückgabe) vereinfachen würden.

Automatische Bestandserfassung. Die «real-time event inventory application» von rapidstock ermittelt den Inhalt offener Flaschen über eine Bluetooth-Waage und plant das Inventar anhand der erwarteten Gästezahl [2]. Ein verwandter Ansatz ist das Zählen per Bilderkennung. Dies wurde für die Schlussinventur des Rattenfests bereits getestet, lieferte aber zu ungenaue Ergebnisse. Auch Starbucks stellte ein vergleichbares System nach wenigen Monaten wieder ein [3]. Für das Rattenfest bringen beide Ansätze wenig: Das Lager gibt ganze Gebinde aus, deren Zählung heute kein Problem ist. Die Gästezahl ist kaum planbar, da rund 60-70 % der Tickets erst am Festtag verkauft werden.

Bezahlung bei der Bestellung. Zahlungslösungen wie SumUp mit kontaktloser Zahlung per NFC [4] oder TWINT mit QR-Code [5] würden es erlauben, Getränke direkt bei der Bestellung zu bezahlen. Für das Rattenfest ist das nicht sinnvoll, da die Bars ihren Getränkebezug nicht vorfinanzieren sollen. Die Einnahmen der Bars fließen zuerst an das Rattenfest, das nach dem Fest den Getränkebezug abzieht und die Differenz auszahlt. Die Bezahlung ist damit Teil der Abrechnung (FR-26) und nicht des Bestellprozesses, eine Zahlungsintegration wird nicht umgesetzt.

Rückmeldung an die Nutzenden. SumUp bestätigt eine Zahlung mit einem Signalton und vier LEDs [4]. Die Bestätigung wird wahrgenommen, ohne dass eine Nachricht gelesen werden muss. Unter Festbedingungen (dunkel, laut, Zeitdruck) ist das besonders wertvoll. Da ein Ton im Lärm untergehen kann, sollte er mit gut sichtbaren Farben kombiniert werden.

Fazit. Die automatische Bestandserfassung löst ein Problem, das am Rattenfest nicht besteht, und rechtfertigt den Aufwand nicht. Auf eine Zahlungsintegration wird bewusst verzichtet, da die Bezahlung über die Abrechnung nach dem Fest erfolgt. Übernommen wird das Prinzip der Rückmeldung über mehrere Kanäle (Ton und Farbe), das in NFR-03 einfließt. Eine automatische Erfassung könnte sich lohnen, falls Bars ihre Getränke künftig selbst aus dem Kühlwagen holen (siehe Abschnitt 8.2).

Webshop für Getränke

Ein Webshop der Getränke verkauft ermöglicht eine gute Benutzerfreundlichkeit durch farbige und auffallende Bilder. Ausserdem gibt es einen Filter für die Getränke, ebenfalls mit einem Bild. Einzelne Getränke und 6er Packs werden in dieser Anzahl auf dem Bild abgebildet. [6]

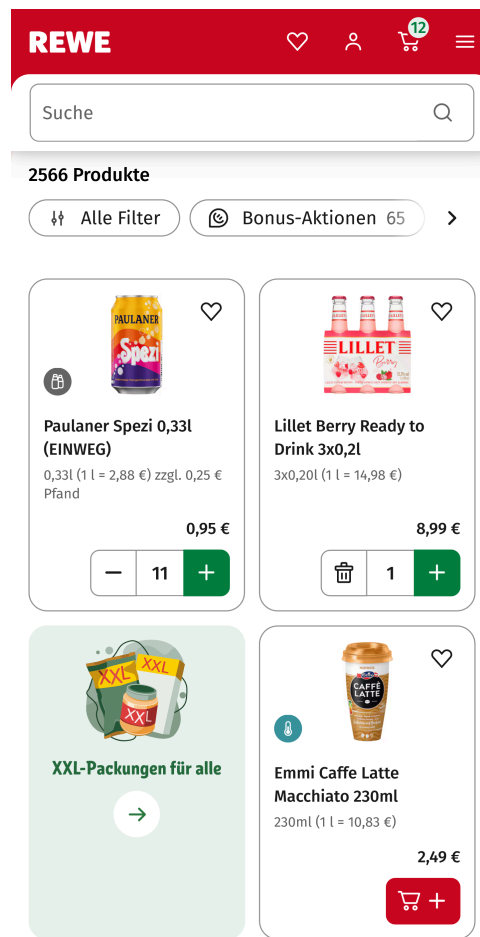


Abbildung 6: Screenshot von REWE Webshop

Bestell-App für Zuschauer im Fussballstadion

Das Fussballstadion hat einige ähnliche Probleme: es ist laut, es hat sehr viele Menschen, die Zuschauer haben wenig Zeit und Getränke werden bestellt, welche möglichst schnell bereit sein sollen. Ausserdem kann die Internetverbindung bei vielen Menschen negativ beeinflusst werden. Wir möchten herausfinden, wie schnell und wie die vorhandenen Lösungen die Besteller informieren und wie die Bestellung ausgeführt wird. Die Bezahlung und Verkaufssteigerung interessiert uns nicht.

Es gibt sehr viele Apps, die bereits das Problem lösen wollten und in Nachrichten erschienen. Hier eine Sammlung mit ungefähren Jahreszahlen: Ein «Call-a-Ber»-Szenario« und ungenügende mit der Mobilfunktechnik im Stadion wurden in 2011 berichtet [7]. Bereits 2013 gab es die Idee, einer App um Getränke per Smartphone zu bestellen [8].

Im englischsprachigen Raum hat 2015 VenueNext ihre App die einen Bestellservice enthält, veröffentlicht [9]. Heute gehört es zu «Shift4» und es wird besonders das einheitliche Bezahlungssystem hervorgehoben [10].

Eine App die sogenannte «Runner» informiert um die Bestellung auszuliefern in 2018 [11]. Eine App «OrdiFy» die den Sitzplan einberechnet in 2021 [12], [13]. Im Februar 2025 wurde dies geschrieben: «Die neue Funktion in der BVB-App soll es den Nutzern ermöglichen, „an Heimspieltagen über die App Getränke und Snacks zu bestellen und auch zu bezahlen“, wie der Fußball-Bundesligist aus Dortmund auf wa.de-Anfrage mitteilt» [14]

Heute arbeiten einige Stadions mit Takeaway Apps zusammen, beispielsweise der Kybunpark und «UEFA Women's EURO 2025» mit Just Eat [15], [16]. Die Bestellung wird dann an dafür vorgesehenen Orten abgeholt.

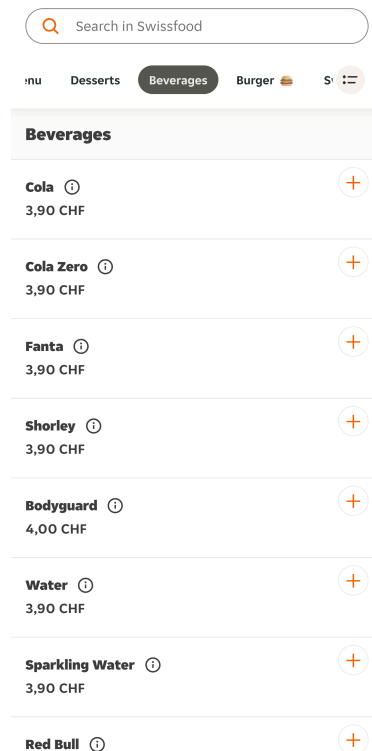


Abbildung 7: Screenshot von just-eat.ch

Die digitale Bestellplattform «Jamezz» verspricht, dass mithilfe eines QR-Codes die Bestellung geöffnet werden kann und der Sitzblock direkt mit dem QR-Code mitgeliefert wird. [17] Die Benachrichtigung an den Besteller löst dieser Service mit E-Mail und SMS, es sei auch eine Echtzeit-Bestellverfolgung mit «in Vorbereitung», «abholbereit», «unterwegs» möglich. [18] Das UI differenziert sich nicht von einem Takeaway Website oder Restaurant Menüplan und ist sehr generisch.

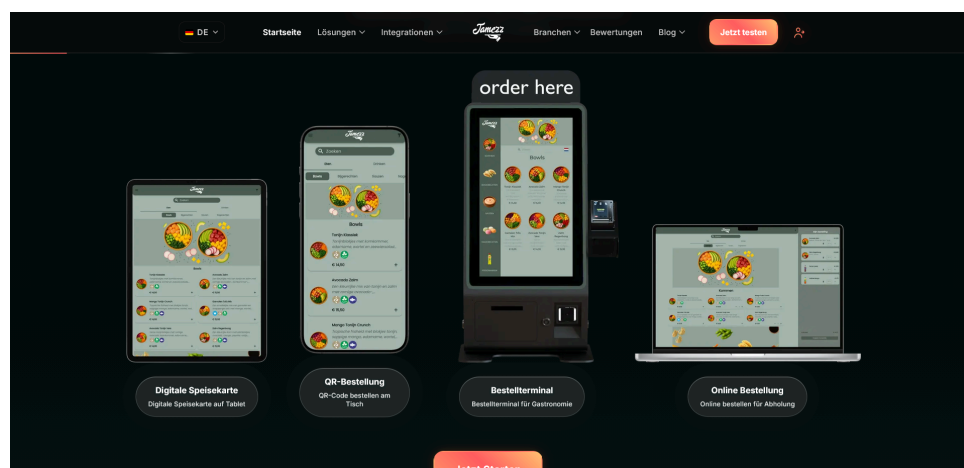


Abbildung 8: Screenshot von jamezz.com/de

Fazit

Diese Lösungen sind sehr an den Verkauf und die Bezahlung ausgerichtet. Heutzutage ist mehr mit Webseiten und Web Apps möglich, so ist auch Just Eat ohne App nutzbar. Die Benachrichtigungen

auf SMS zu erweitern, könnte die schlechte Internetverbindung ausgleichen. Neben der zusätzlichen Komplexität und Kosten müsste dafür jeder Bar-MA sein eigenes Profil mit Telefonnummer speichern. Mit AWS End User Messaging wäre der Preis pro SMS ohne weitere Einrichtungskosten \$0.05124 [19].

3.2 Nutzungskontext am Fest

Restaurant und Bars Umgebung

Die Umgebung am Rattenfest ist ähnlich zu der in einer Küche von einem Restaurant oder einer vollen Bar.

Bestellung von Restaurant https://is.muni.cz/th/yzp5k/Study_on_home_delivery_platforms.pdf

Ablenkung im Restaurant Ablenkung <https://firstmonday.org/ojs/index.php/fm/article/view/6949/5629> «cognitive overload and the erosive lack of focus»

Stress minimieren <https://accesson.kr/ijcon/v.21/3/1/57459> «4 different categories (User Experience, Usability, Psychophysiology of Stress and Interface Visual Design) were identified, defined and exemplified: Content Overload, Service Instability, Low level of Attractiveness, Lack of Control, Low level of Safety, Low Usability, Unpredictability, Uncertainty, Unfamiliarity, Judgment or social evaluation threat and Poor Visual Design. This set of elements could help design better interfaces while taking into consideration users' mental and emotional state.»

Einfluss auf die Umgebung auf die Bar Mitarbeiter, ist besonders die kürzere Aufmerksamkeitsspanne durch die vielen Ablenkungen, kürzeres Kurzzeitgedächtnis und höherer Stress. Gemäss ^ sind die wichtigsten Stressfaktoren «Content Overload, Service Instability, Low level of Attractiveness, Lack of Control, Low level of Safety, Low Usability, Unpredictability, Uncertainty, Unfamiliarity, Judgment or social evaluation threat and Poor Visual Design».

Wir suchen ein gutes Mass zwischen «Content Overload», zuviele Informationen die den Nutzer überfordern und zu wenige Informationen. Ein Bar-Mitarbeiter soll, nach einer kurzen Ablenkung, dort weitermachen können wo er aufgehört hat. Die weiteren Punkte .. können durch User Experience Tests des fertigen Produktes gelöst werden. NFR... Usability

Accessibility Guidelines übertragen

Ein schlechtes Beispiel: der Bar Mitarbeiter erhält den Auftrag, das Getränk XY nachzubestellen, dann öffnet dieser den Link zur Website und sucht den Button «neue Bestellung» und klickt darauf. Sofort kommt jemand und sagt ob er schnell ein Becher von XY abfüllen kann. Danach soll er fortsetzen aber jetzt sieht er eine Liste von ganz vielen Getränkeartikeln und weiss nicht mehr, wie er dahin gekommen ist und was er machen wollte. Er muss nochmals den Link öffnen und kann dieses Mal die Bestellung erstellen. Er klickt aber auf das falsche Getränk und da es schnell gehen muss hat er die Bestellung bereits abgeschickt.

Die Nutzer unserer Website, sind aber nicht betrunken, doch kann die spezielle Umgebung das Bewusstsein verschlechtern und einen Nutzer ablenken. Es kann das Erinnerungsvermögen verschlechtern beispielsweise durch Ablenkungen. Ausserdem gibt es Zeitdruck an einer Bar, besonders bei vielen Besuchern die etwas kaufen oder schnell ihren Becher zurückgeben wollen, was auch zu Fehlclicks führen kann.

Wir übertragen Accessibility Guidelines von älteren Menschen auf unsere Bar Mitarbeiter, aufgrund der speziellen Umgebung.

Wichtige Faktoren für gute Usability für ältere Personen für eine Web-Lösung sind erstens ein klarer Entscheidungsprozess mit klaren Schritten die an die Prioritäten vom Nutzer angepasst werden.

Zweitens soll der Inhalt einfach gehalten werden, dass heisst einfache Sprache und Piktogramme nutzen. Drittens soll das UI intuitiv und benutzerfreundlich sein, Informationen sollen auf ein minimum reduziert werden (cognitive overload). Es kann Pop-Up Fenster und Informationen/Tipps anzeigen. Drop-down Menüs und Scrolling soll möglichst verhindert werden. [20]

Den zweiten Punkt mit der einfachen Sprache nennt sich auch «cognitive accessibility» und gehört zu den zusätzlichen Standards von WCAG. Die wichtigsten Regeln sind einfache Wörter, Präsensform, kurze Sätze und kurze Textblöcke, unmissverständlichen Inhalt, klare Bilder und einfache Videos. Ausserdem soll man sich nicht auf das mathematische Können der Nutzer verlassen. [21]

Die «W3C Web Accessibility Initiative (WAI)» entwickelt weiter Standarde und Hilfen um die Accessibility (Zugänglichkeit) sicherzustellen [22]. Für uns sind die «Web Content Accessibility Guidelines (WCAG)» [23] relevant, aber User Agent Accessibility Guidelines (UAAG), WAI-ARIA und Authoring Tool Accessibility Guidelines (ATAG) weniger.

Für die WCAG gibt es eine Quick Reference [24]. Wir haben einige Regeln für uns übernommen und in der Checkliste auf WCAG verlinkt.

Einhändige Nutzung

Die Bar MA könnten nur eine Hand frei haben, beispielsweise beim nachschauen ob die Bestellung bereit ist. Deshalb sollen diese Aktionen so auf dem Bildschirm positioniert sein, damit sie gut mit dem Daumen erreicht werden. Weniger häufig genutzte Aktionen wie die Einstellungen können weiter weg platziert werden.

Guidelines für Rattenfest Website

Eine Zusammenfassung der oben besprochenen Accessibility Guidelines, angepasst auf die Umgebung am Rattenfest. Auf der verlinkten WCAG Website finden sich weitere Informationen.

Die Punkte wurden zusätzlich in passende Untertitel gruppiert, die sinnvoll sind für diese Punkte.

klarer Entscheidungsprozess (Bestellung aufnehmen, Vorbestellung)

- ☐ Klare Schritte aufzeigen
- ☐ jeder Schritt ist sinnvoll getrennt und verständlich
- ☐ In der Navigation ist der aktuelle Schritt, vorherige und nächste Schritt ersichtlich
 - siehe auch <https://www.w3.org/WAI/WCAG22/quickref/#location>
- ☐ Der Nutzer wird informiert, wenn eine Aktion den Kontext wechselt (Seitenwechsel, vorhandene Optionen werden geändert) <https://www.w3.org/WAI/WCAG22/quickref/#on-input>

Einheitlichkeit

- ☐ Die Navigationssteuerung ist einheitlich <https://www.w3.org/WAI/WCAG22/quickref/#consistent-navigation>
- ☐ Komponente mit gleichen Funktionalitäten werden einheitlich dargestellt und genutzt <https://www.w3.org/WAI/WCAG22/quickref/#consistent-identification>
- ☐ Kontextänderungen werden nur durch den Nutzer aufgerufen, oder können durch den Nutzer deaktiviert werden <https://www.w3.org/WAI/WCAG22/quickref/#change-on-request> (Beispielsweise wenn der Nutzer auf der Website ist und eine Nachricht erhält, soll entweder nicht ein Popup den Bildschirm damit verdecken oder der Nutzer kann diese Popups deaktivieren)

einfache Sprache

- ☐ einfache Wörter

- ☐ Präsensform
- ☐ kurze Sätze und kurze Textblöcke
- ☐ unmissverständlichen Inhalt
- ☐ klare Bilder und einfache Videos
- ☐ Text ist nicht länger als 80 Zeichen pro Zeile

Informationen

- ☐ Piktogramme, Bilder, Visualisierungen sinnvoll eingesetzt
- ☐ Informationen auf ein Minimum reduzieren
- ☐ Keine Rechnungen im Kopf vom Nutzer nötig

Navigation und Bedienung

- ☐ Es ist möglichst kein Scrolling notwendig
- ☐ Es gibt keine Drop-Down Menüs
- ☐ Inhalt ist nicht hinter Hover/Focus versteckt
- ☐ Es gibt keine zeitbegrenzten Optionen oder der Nutzer kann den Timer deaktivieren. Ausser es ist an eine Real-Time Aktivität gebunden, beispielsweise die Bestellung ist bereits bereitgestellt worden. <https://www.w3.org/WAI/WCAG22/quickref/#timing-adjustable>

Aktionen ausführen/ Buttons und Links / Operable / Bedienbarkeit (aber ohne Bedienung oben ^)

- ☐ Alles was mit Gestensteuerung möglich ist, kann auch durch Klicks erreicht werden <https://www.w3.org/WAI/WCAG22/quickref/#pointer-gestures>
- ☐ Die Grösse von Targets (Buttons, Links, alles was anklickbar ist) sind mindestens 44 x 44 Pixel gross (Ausnahmen dokumentiert erlaubt) <https://www.w3.org/WAI/WCAG22/quickref/#target-size-enhanced>
- ☐ Input ist mit einer Computermouse und Touchscreen möglich <https://www.w3.org/WAI/WCAG22/quickref/#concurrent-input-mechanisms>
- ☐ Es werden keine Drag & Drop oder Mouse-Down Events genutzt.

Eingabehilfen

- ☐ Die Aktion von einem Link und Button ist eindeutig erkennbar <https://www.w3.org/WAI/WCAG22/quickref/#link-purpose-in-context>
- ☐ Input Fehler werden dem Nutzer sinnvoll als Text erklärt <https://www.w3.org/WAI/WCAG22/quickref/#error-identification>
- ☐ Wenn möglich werden bei Input Fehlern Empfehlungen/Vorschläge angezeigt <https://www.w3.org/WAI/WCAG22/quickref/#error-identification>
- ☐ Bei erfolgreicher Eingabe erhält Nutzer eine Erfolgsmeldung
- ☐ Nachrichten wie eine Erfolgsmeldung oder Bestellnachrichten sollten nicht automatisch verschwinden, da es nicht sichergestellt ist, dass der Nutzer nicht wegschaut.

Zusätzliche Hilfen

- ☐ Informationen und Tipps als Pop-Up auffindbar
- ☐ Hilfen sind einheitlich platziert <https://www.w3.org/WAI/WCAG22/quickref/#consistent-help>

Lesbarkeit in speziellen Lichtverhältnissen

- ☐ Passende Farbwahl <https://www.w3.org/WAI/WCAG22/quickref/#use-of-color>
- ☐ Kontrastverhältnis mindestens 4.5:1 wo möglich 7:1
- ☐ Zoomen soll möglich sein <https://www.w3.org/WAI/WCAG22/quickref/#resize-text>

- ☐ Es befindet sich kein Text in den Bildern <https://www.w3.org/WAI/WCAG22/quickref/#images-of-text>

Sitzung

- ☐ Daten sind gespeichert, wenn Nutzer sich neu authentifizieren musste <https://www.w3.org/WAI/WCAG22/quickref/#re-authenticating>

Position der Bedienungselemente

- ☐ Häufig nutzbare Aktionen befinden sich nahe beim Daumen (unten) Abschnitt 3.2.2.1

Die Nutzung soll auf Smartphone und Laptops bedienbar sein. Der Prototyp der Vorarbeit wurde sogar von einer Bar mit einem Laptop genutzt, wobei die meisten das private Smartphone nutzten.

Die Tastaturnavigation wird nicht speziell geprüft. Es wird semantisches HTML genutzt womit dies möglichst sichergestellt wird. Auch spezifische Guidelines für Screen-Reader sind nicht in der Checkliste enthalten, da sie für die Zielgruppe irrelevant sind. So wurde auch «2.4.2 Page Titled» nicht als Bedingung notiert, um Freiheit zu lassen beim Seitenaufbau und da die Bedingung «In der Navigation ist der aktuelle Schritt, vorherige und nächste Schritt ersichtlich» bereits diesen Punkt erfüllt.

Das Kontrastverhältnis sollte wo möglich 7:1 betragen, darf aber an speziellen Orten 4.5:1 sein. Beim Prototyp aus der Vorarbeit genügt der Kontrast knapp, es hat grauen Text (#737373) auf hellgrünem Hintergrund (#F0FDF4) mit dem Kontrastverhältnis 4.52:1. Es soll auch die Schriftgrösse einbezogen werden bei der Überlegung, ob der Kontrast genügt.

Das Dashboard mit den Bestellungen wird automatisch mit neuen Bestellungen aktualisiert. Wir entscheiden uns jedoch dafür, dass die Updates nicht pausiert werden können. <https://www.w3.org/WAI/WCAG22/quickref/#pause-stop-hide>

Die Animationen können nicht deaktiviert werden. Es wird darauf geachtet, dass die Animationen den durchschnittlichen Nutzer nicht stören. <https://www.w3.org/WAI/WCAG22/quickref/#animation-from-interactions>

Der Punkt «2.4.4 Link Purpose (In Context)» wurde erweitert zu Buttons (<https://www.w3.org/WAI/WCAG22/quickref/#link-purpose-in-context>).

<https://www.w3.org/WAI/WCAG22/quickref/#contrast-enhanced> <https://www.w3.org/WAI/WCAG22/quickref/#contrast-minimum>

Einige Punkte müssen bereits vor dem Programmieren beachtet werden. Dazu gehört die Zoom-Funktionalität.

Dark Mode

Da es spezielle Lichtverhältnisse gibt am Rattenfest, wollten wir herausfinden ob sich ein Dark Mode besser eignet.

Gemäss [25] macht bei schwachem Licht Dark-Mode, im Gegensatz zum Light-Mode, das Auge eher müde. Es wird aber auch erwähnt, dass andere Studien zu einem anderen Ergebnis geführt haben, was an der Bildschirmgrösse liegen könnte [26].

Ausserdem ist der Prototyp der am letzten Rattenfest getestet wurde Light-Mode und es gab keine spezifischen Wünsche für den Dark-Mode.

Deshalb wird die Website im Light Mode entwickelt.

3.3 Erkenntnisse

4 Anforderungen

4.1 Akteure

- **Bar-Team:** Gibt eine Vorbestellung ab und bestellt während dem Fest Getränke für seine Bar und holt sie ab.
- **Lagerteam:** nimmt während dem Fest Bestellungen entgegen, stellt sie bereit und übergibt sie.
- **Getränkchef (RF-OK):** verwaltet Festaussage, Benutzer, Sortiment, Bars und Abrechnung.

4.2 User Stories und Functional Requirements

Vorbereitung

US-01: Als RF-OK möchte ich eine neue Festaussage anlegen, damit die Daten jedes Jahrgangs getrennt aber vergleichbar sind.

- **FR-01:** Das System muss erlauben, eine Festaussage anzulegen und als aktiv zu setzen. Sortiment, Bestände, Bestellungen und Rückgaben sind jeweils einer Festaussage zugeordnet.
- **FR-02:** Das System muss erlauben, Sortiment und Bars einer früheren Festaussage als Ausgangslage zu übernehmen.

US-02: Als RF-OK möchte ich die Bars erfassen und ihnen Zugänge geben, damit Bestellungen eindeutig zugeordnet sind.

- **FR-03:** Das System muss erlauben, Bars mit Bezeichnung und Kontaktangabe zu erfassen, zu ändern und zu deaktivieren.
- **FR-04:** Das System muss pro Bar einen Zugang als teilbaren Link erzeugen und erlauben, diesen neu zu erzeugen, womit der bisherige ungültig wird.

US-03: Als RF-OK möchte ich das Getränkesortiment unseres Lieferanten den Bars zur Verfügung stellen.

- **FR-05:** Das System muss erlauben, Getränke mit Bezeichnung, Gebindegrösse und Preis zu erfassen.
- **FR-06:** Das System muss erlauben, das Getränkesortiment aus einer Datei zu importieren.

US-04: Als Bar-Team möchte ich meinen Bedarf vor dem Fest schätzen, damit die benötigte Ware eingekauft wird.

- **FR-07:** Das System muss erlauben, pro Bar eine Bedarfsschätzung über das Sortiment zu erfassen und bis zu einem Stichtag zu ändern.

US-05: Als RF-OK möchte ich die Schätzungen prüfen und zusammenfassen, damit ich daraus die Bestellung beim Getränkehändler ableiten kann.

- **FR-08:** Das System muss die Schätzungen aller Bars darstellen und einzeln korrigierbar machen.

US-06: Als RF-OK möchte ich die gelieferte Ware einlagern, damit der Lagerbestand von Beginn an stimmt.

- **FR-09:** Das System muss den Anfangsbestand aus der Bestellung beim Händler übernehmen.

Bestellung während des Fests

US-07: Als Bar-Team möchte ich online Getränke bestellen.

- **FR-10:** Das System muss erlauben, eine Bestellung über mehrere Artikel mit Mengenangabe abzuschicken.
- **FR-11:** Das System darf nur Mengeneingaben erlauben die den Gebindegrössen entsprechen.
- **FR-12:** Das System muss verhindern, dass mehr bestellt wird, als im Lager verfügbar ist.

- **FR-13:** Das System muss einen Hinweis anzeigen, wenn eine Bestellung einen Artikel enthält, der sich von der Reservation der Bar unterscheidet.

US-08: Als Bar-Team möchte ich eine abgesendete Bestellung stornieren können.

- **FR-14:** Das System muss erlauben, eine Bestellung zu stornieren, solange das Lagerteam noch nicht bearbeitet hat.

US-09: Als Bar-Team möchte ich eine frühere Bestellung wiederholen.

- **FR-15:** Das System muss die Bestellhistorie der eigenen Bar anzeigen und erlauben, einen früheren Eintrag als Vorlage zu übernehmen und vor dem Absenden anzupassen.

Bearbeitung und Abholung

US-10: Als Lagerteam möchte ich eingehende Bestellungen umgehend sehen, damit ich ohne Verzögerung mit dem Bereitstellen beginnen kann.

- **FR-16:** Das System muss Bestellungen und deren Statusänderungen bei Bar und Lagerteam ohne manuelles Neuladen aktualisieren.
- **FR-17:** Das System muss offene Bestellungen in der Reihenfolge ihres Eingangs darstellen.

US-11: Als Bar-Team möchte ich zeitnah Bescheid erhalten, sobald meine Bestellung bereitsteht.

- **FR-18:** Das System muss erlauben, eine Bestellung als abholbereit zu markieren.
- **FR-19:** Das System muss erlauben, die Abholung zu bestätigen, womit die Bestellung abgeschlossen wird.
- **FR-20:** Das System muss eine Benachrichtigung an das Bar-Team senden, wenn die Bestellung abholbereit ist.

US-12: Als Lagerteam möchte ich Abweichungen festhalten, damit sie bei der Abrechnung nachvollziehbar sind.

- **FR-21:** Das System muss erlauben, eine Bestellung mit Begründung zu stornieren oder als nicht abgeholt zu kennzeichnen.

US-13: Als Bar-Team möchte ich bei technischen Problemen oder Zwischenfällen jemanden erreichen.
Abschnitt 2.1.2

- **FR-22:** Das System muss eine Kontaktmöglichkeit zum Lagerteam und Sicherheitsverantwortlichen anzeigen.

US-14: Als Lagerteam möchte ich eine Pause einlegen, damit die Bars wissen, dass Bestellungen vorübergehend nicht bearbeitet werden. Abschnitt 2.1.2

- **FR-23:** Das System muss erlauben, das Lager als vorübergehend nicht besetzt zu markieren, was den Bars angezeigt wird.

Rücknahme und Abrechnung

US-15: Als Bar-Team möchte ich nicht verkaufte Ware zurückgeben, damit sie mir nicht verrechnet wird.

- **FR-24:** Das System muss erlauben, eine Rückgabe analog zu einer Bestellung zu erfassen, begrenzt auf die von der Bar bezogenen Artikel und Mengen.
- **FR-25:** Das System muss verlangen, dass das RF-OK eine Rückgabe bestätigt, bevor sie dem Lagerbestand gutgeschrieben wird.

US-16: Als RF-OK möchte ich nach dem Fest einfach und schnell pro Bar abrechnen.

- **FR-26:** Das System muss pro Bar eine Aufstellung aller bezogenen und zurückgegebenen Artikel mit Betrag erzeugen.
- **FR-27:** Das System muss Abrechnung und Rohdaten der Bezüge als Datei exportieren.

US-17: Als RF-OK möchte ich den Verbrauch mehrerer Feste vergleichen, damit die Schätzung im Folgejahr besser wird.

- **FR-28:** Das System muss die Daten vergangener Feste erhalten und pro Artikel vergleichbar darstellen.

4.3 Priorisierung

Die Functional Requirements sind nach der MoSCoW-Methode in drei Kategorien eingeteilt.

Kategorie	Bedeutung
Muss	Ohne diese Anforderungen erfüllt das System seinen Zweck nicht.
Soll	Wichtig für Bedienbarkeit, Effizienz oder Zuverlässigkeit. Das System funktioniert ohne sie, aber mit geringerer Qualität.
Kann	Wünschenswerte Ergänzung, die entfällt, wenn die Zeit nicht reicht.

Tabelle 2: Kategorien der MoSCoW-Klassifikation

ID	Kurzbezeichnung	Priorität
FR-01	Festausgabe anlegen und aktiv setzen	Muss
FR-02	Stammdaten aus Vorjahr übernehmen	Soll
FR-03	Bars verwalten	Muss
FR-04	Bar-Zugang erzeugen und erneuern	Muss
FR-05	Sortiment verwalten	Muss
FR-06	Sortiment importieren	Kann
FR-07	Bedarfsschätzung erfassen	Muss
FR-08	Schätzungen aggregieren und korrigieren	Muss
FR-09	Anfangsbestand erfassen	Muss
FR-10	Bestellung absenden	Muss
FR-11	Schrittweite für Mengeneingabe	Soll
FR-12	Bestand als Obergrenze	Muss
FR-13	Hinweis bei auffälliger Bestellung	Kann
FR-14	Bestellung stornieren (Bar)	Muss
FR-15	Historie anzeigen und wiederverwenden	Soll
FR-16	Aktualisierung ohne Neuladen	Muss
FR-17	Offene Bestellungen nach Eingang	Muss
FR-18	Als abholbereit markieren	Muss
FR-19	Abholung bestätigen	Muss
FR-20	Benachrichtigung ausserhalb der App	Kann
FR-21	Bestellung kennzeichnen (Lager)	Soll
FR-22	Kontaktmöglichkeit	Soll
FR-23	Lager als nicht besetzt markieren	Kann
FR-24	Rückgabe erfassen	Soll
FR-25	Rückgabe bestätigen	Soll
FR-26	Abrechnung pro Bar	Muss
FR-27	Datenexport	Soll
FR-28	Vergleich über Festausgaben	Kann

Tabelle 3: Klassifikation der Functional Requirements

4.4 Non-Functional Requirements

Die folgenden Anforderungen gelten für das System als Ganzes. Zu jeder ist angegeben, auf welche Functional Requirements sie sich bezieht und woran ihre Erfüllung gemessen wird. Da das Rattenfest nach Abgabe dieser Arbeit stattfindet, sind alle Anforderungen ohne das Fest überprüfbar. Im Vordergrund stehen die Zuverlässigkeit während des Fests und die Betreibbarkeit über mehrere Jahre. Skalierbarkeit, Redundanz und Barrierefreiheit sind bewusst niedrig angesetzt, da das System nicht öffentlich zugänglich ist und von weniger als 20 gleichzeitig verbundenen Personen genutzt wird.

NFR-01 (Konsistenz der Bestandsführung) betrifft FR-10, FR-12, FR-24

Eine Bestellung wird vollständig oder gar nicht verbucht, und auch bei wiederholtem Absenden genau einmal. Bei gleichzeitigen Bestellungen mehrerer Bars auf denselben Artikel darf die Summe der bestätigten Bezüge den verfügbaren Bestand nicht überschreiten und der Bestand nicht negativ werden.

Messung: Manueller Test, der zwei Bestellungen auf einen Artikel mit Bestand 1 gleichzeitig absetzt: genau eine wird bestätigt, die andere abgelehnt, keine verbleibt in einem Zwischenzustand. Zusätzlich wird dieselbe Bestellung mehrfach abgesendet und darf nur einmal im Bestand erscheinen.

NFR-02 (Latenz der Statusaktualisierung) betrifft FR-16, FR-18

Eine Statusänderung ist beim jeweils anderen Akteur innerhalb von zwei Sekunden sichtbar (95. Perzentil), bei 20 gleichzeitig verbundenen Bar-Geräten.

Messung: Lasttest mit simulierten Clients, gemessen wird die Zeit zwischen dem Auslösen der Statusänderung und ihrem Eintreffen beim Empfänger.

NFR-03 (Bedienbarkeit unter Festbedingungen) betrifft FR-10, FR-15

- Benachrichtigungen mit «Beep» und Farben nutzen

Messung:

NFR-04 (Geräte- und Bildschirmunterstützung)

Alle Funktionen sind auf aktuellem Chrome (Android), Safari (iOS) und einem Desktop-Browser bei Bildschirmbreiten von 320 bis 1920 Pixeln vollständig nutzbar.

Messung: Manuelle Testmatrix über die definierten Kombinationen aus Browser, Betriebssystem und Bildschirmbreite.

NFR-05 (Nachvollziehbarkeit der Abrechnung) betrifft FR-21, FR-26, FR-27

Jede Bestandsänderung, also Bezug, Rückgabe und Korrektur, ist dauerhaft mit Zeitpunkt, Bar und auslösendem Benutzer gespeichert, sodass jeder Betrag der Abrechnung auf einzelne Vorgänge zurückführbar ist.

Messung: Die Abrechnung einer Testbar wird stichprobenweise auf die zugrunde liegenden Einzelsvorgänge zurückgerechnet.

NFR-06 (Betreibbarkeit durch das OK) betrifft FR-01 bis FR-05, FR-09

Alle wiederkehrenden Betriebsaufgaben, also das Anlegen einer Festausgabe, die Verwaltung von Bars und Sortiment, Bestandskorrekturen und die Abrechnung, sind über die Oberfläche ausführbar und erfordern keinen direkten Datenbankzugriff. Eine nicht am Projekt beteiligte Person richtet anhand der Anleitung eine neue Festausgabe in unter 30 Minuten ein.

Messung: Nachweis der Oberflächenabdeckung anhand der Liste der Betriebsaufgaben, Durchführung der Einrichtung durch eine projektfremde Person.

NFR-07 (Zugriffsschutz der Bar-Zugänge) betrifft FR-04

Ein Bar-Zugang ist nicht erratbar (mindestens 122 Bit Entropie) und gewährt ausschliesslich Zugriff auf die Daten der eigenen Bar. Administrative Funktionen sind über diesen Zugang nicht erreichbar.

Messung: Negativtests gegen fremde Bar- und Administrationsendpunkte mit einem gültigen Bar-Zugang.

5 Design und Architektur

5.1 Technologieentscheid

Kriterien des Frameworks

1. einfache Infrastruktur
 - 1a? zentrale Möglichkeit zum Testen und Updaten für Nicht-Informatiker möglich
 - 1b? langlebig/bewährt,
 - 1c? Unabhängigkeit vom Framework, erleichtert das Wechseln auf ein anderes Framework (Zusammenhang zu «einfach zu updaten»)
2. PWA mit Benachrichtigungen möglich
3. Geeignet für kleine/mittelgrosse Website
4. Einfacher Einstieg, keine komplexen Konfigurationen nötig, einfach zu lernen

Vorauswahl

Die Performance ist in unserem Fall nicht abhängig vom Framework, sondern vom Hosting-Provider. Der Unterschied zwischen React und Angular beträgt nur Millisekunden [27]. Ausserdem verbessern die Frameworks ihre Performance laufend [27].

Wir suchen eine Lösung mit Client-Side Rendering, damit neuer Inhalt ohne ein Neuladen angezeigt wird.

Für das Hosting kennen wir bereits Vercel und Supabase für die Datenbankanbindung, dieses eignet sich für React, Vue und Angular.

Es wurden die 4 momentan grössten Frontend Web Frameworks analysiert und nach den Kriterien verglichen [28]. Diese Frameworks besitzen eine gute Dokumentation und viele Nutzer.

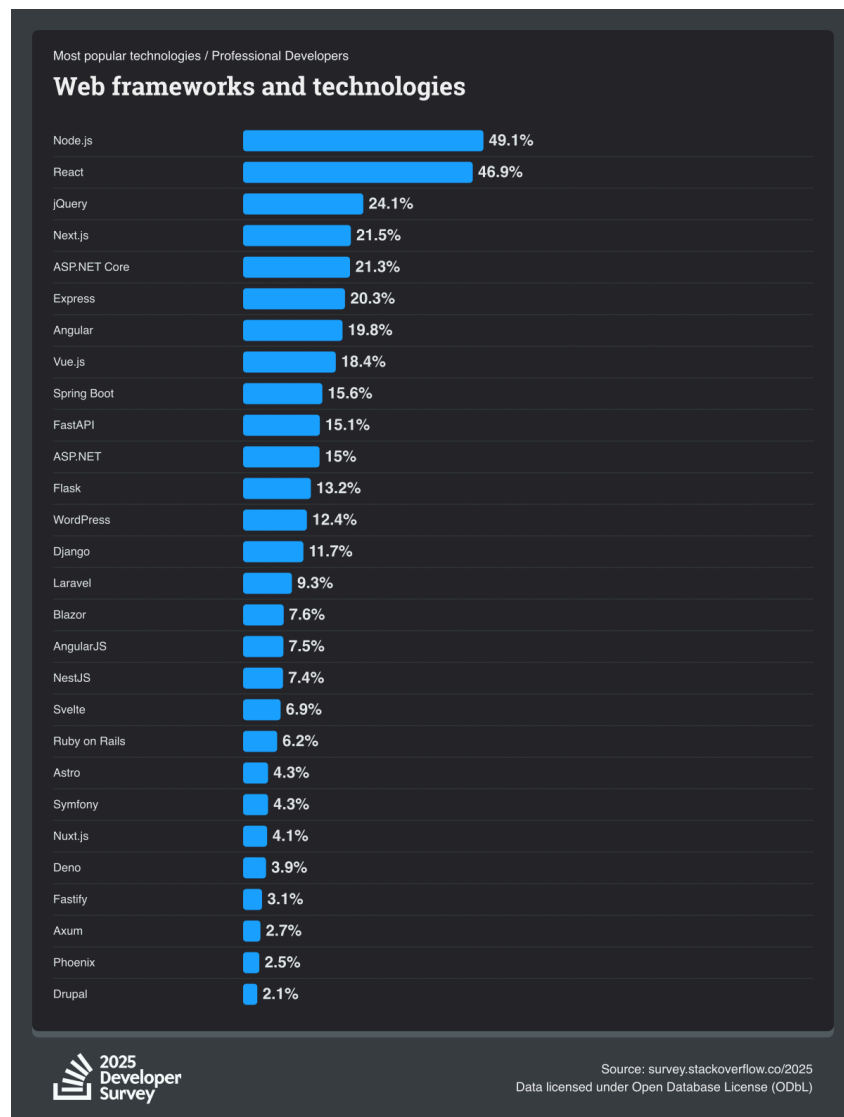


Abbildung 9: Stack Overflow Umfrage 2025

<https://2025.stateofjs.com/en-US/libraries/>

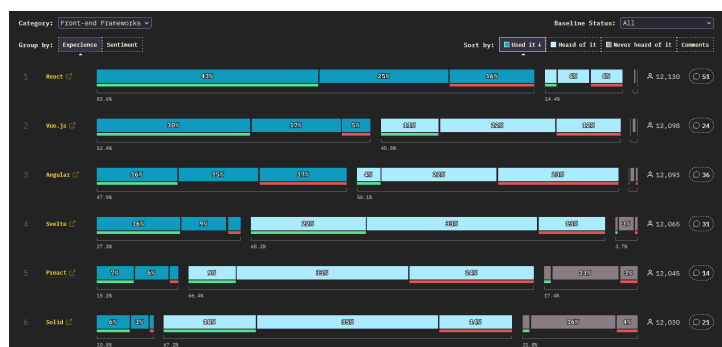


Abbildung 10: Beschreibung

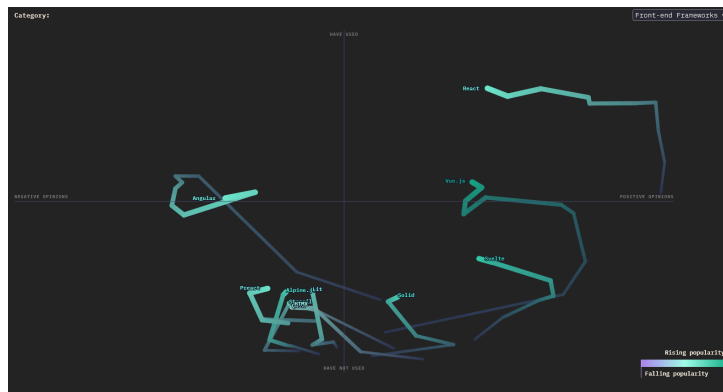


Abbildung 11: Beschreibung

Informationen

React / Next.js mit Typescript

1. Aufwändigere erste Einrichtung, Testing und Builder muss manuell eingerichtet werden.

Wir können eine Vorlage wie den T3Stack verwenden, durch die vielen Abhängigkeiten steigt aber die Komplexität und Aufwand beim Updaten. Ausserdem wird dann Next.JS verwendet. [29]

2. PWA ist möglich mit create-react-app (deprecated), Next.JS (übertrieben) <https://www.overcode.tech/blog/react-pwa> oder mit Vite + PWA von Vite (dokumentiert) <https://vite-pwa-org.netlify.app/guide/>
3. React eignet sich gut für diese Projektgrösse. [30]
4. Da wir bereits React kennen ist für uns der Einstieg einfach. Für React muss man JSX verstehen.

Updates

Info: Major Release enthält neue Features und möglicherweise das entfernen von veralteten Funktionen. Es ist möglicherweise nötig, den Code upzudaten, neue Tests zu schreiben und neue APIs zu lernen. Bugfixes und Sicherheitsupdates sind auch im LTS (Long-Term Support) nötig, aber der Code muss nicht verändert werden.

React wurde 2013 veröffentlicht. [31]. Es gibt keine Information in der Dokumentation, wie häufig es einen neuen «Major Release» gibt. Aber React veröffentlichte seit 2020 alle zwei Jahre einen neuen «Major Release» mit «Breaking Changes» [32].

Angular

1. Einfache und schnelle Einrichtung ohne die Datenbankbindung. [33]
 - Der LTS (Long-Term Support) beträgt 12 Monate, dass bedeutet Angular veröffentlicht jedes Jahr einen neuen Major Release. [34]
2. PWA integriert und gut dokumentiert von Angular [35]
3. Angular gemacht für grosse Unternehmens-Systeme. [30]
4. Angular hat eine hohe Lernkurve durch die starke und Meinungsstarke (opinionated) Architektur. [36] Das Vorwissen von React und Typescript macht den Einstieg leichter.

Vue / Vue.js

[37]

1. Einfache Einrichtung und die Tools (JSX Support, Vitest, Router..) können direkt ausgewählt werden.
2. PWA möglich <https://cli.vuejs.org/core-plugins/pwa.html#configuration> <https://www.vuemastery.com/blog/getting-started-with-pwas-and-vue3/#starting-a-pwa-from-scratch>
3. Gemacht für kleine Projekte aber lässt sich auch gut skalieren.
4. Einfach zu lernen, da alles im Framework integriert ist und der Syntax einfacher ist als React. [38]

ASP.NET

Blazor lässt C# Code im Browser in einer WebAssembly .NET Laufzeitumgebung (Runtime) laufen [39]. Jedoch muss es zuerst die WebAssembly Laufzeitumgebung herunterladen, was die Ladezeit im Vergleich zu JS-basierten Webapps verlangsamt [40].

1. Das Setup ist sehr ausführlich erklärt, es wird keine Shell benötigt wie in den anderen Frameworks. [41]
2. Die PWA Einrichtung ist ausführlich dokumentiert. Für die Push-Benachrichtigungen wird ein Backend Server benötigt. [42]
3. Geeignet für grosse Websites und Unternehmen die bereits .NET nutzen.
4. Es wird in C# programmiert und es werden Komponente genutzt ähnlich zu React. [41] Für das HTML wird im Gegensatz zu React, welches JSX nutzt, Razor verwendet [43]. So muss ein zusätzlicher Syntax gelernt werden.

Fazit

5.2 Architektur

5.3 Datenmodell

5.4 UI-Konzept

5.5 Zugangskonzept

6 Implementation

7 Ergebnisse

7.1 Erfüllung der Functional Requirements

7.2 Erfüllung der Non-Functional Requirements

7.3 Usability-Tests

8 Schlussfolgerungen und Ausblick

8.1 Diskussion

8.2 Ausblick

9 Projektmanagement

9.1 Vorgehen

Der Aufwand beträgt 2 Arbeitstage pro Woche für ca. 17h/W , insgesamt 240 Stunden Aufwand.

9.2 Zeitplan und Meilensteine

Hier ist der Zeitplan mit den wichtigsten Meilensteinen.

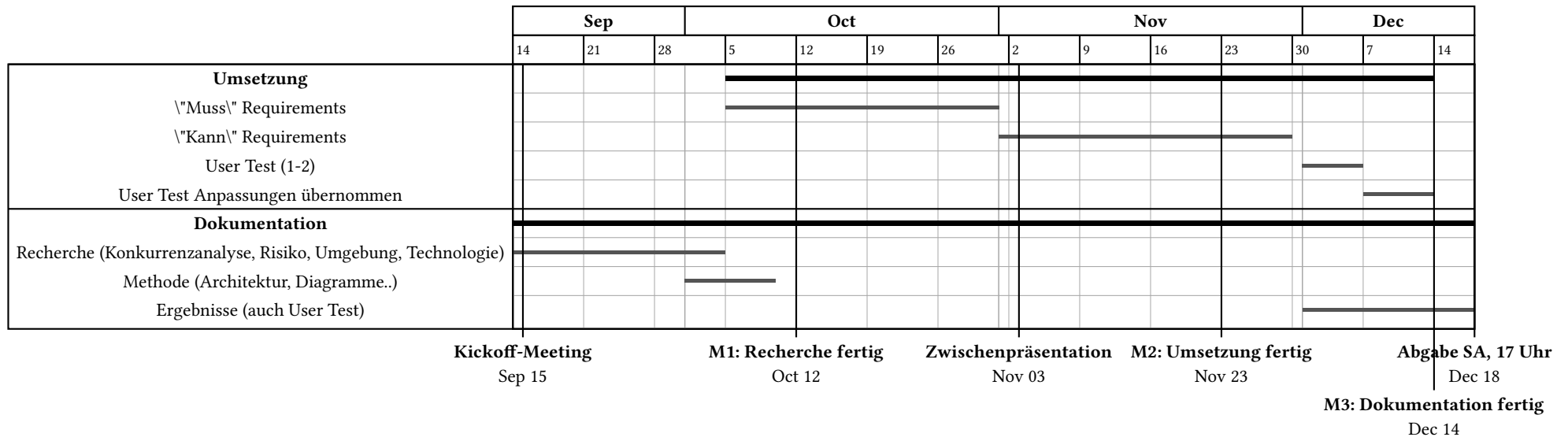


Abbildung 12: Langfristiger Zeitplan

M1: Recherche fertig

Die Recherche (Literatur und Befragung von Usern) wurde ausgewertet und es wurden mindestens drei Beschlüsse daraus gezogen, die sich in den FR oder NFR zeigen.

M2: Umsetzung, Programmierung abgeschlossen

Die Umsetzung der FR und NFR ist abgeschlossen und die Website ist im Internet erreichbar.

M3: Dokumentation bereit zum Korrekturlesen

Die Dokumentation beinhaltet alle nötigen Informationen gemäss dem Leitfaden und ist bereit zum Korrekturlesen.

10 Glossar

Begriff	Definition
Abrechnung	Aufstellung der von einer Bar bezogenen und zurückgegebenen Getränke nach dem Fest. Die Einnahmen der Bars fliessen zuerst an das Rattenfest, das den Getränkebezug abzieht und die Differenz auszahlt.
Anfangsbestand	Lagerbestand zu Beginn des Fests. Er entspricht der beim Getränkehändler bestellten und gelieferten Ware.
Artikel	Ein Getränk im Sortiment mit Bezeichnung, Gebindegrösse und Preis. In der Applikation wird der Begriff Artikel anstelle von Getränk verwendet, da er diese Eigenschaften besser beschreibt. In dieser Dokumentation werden beide Begriffe synonym verwendet.
Bar-Team	Die Mitarbeitenden einer Bar. Sie erfassen die Bedarfsschätzung, bestellen während des Fests Getränke und holen diese im Lager ab.
Bedarfsschätzung	Von einer Bar vor dem Fest erfasste Mengen pro Artikel. Das RF-OK leitet daraus die Bestellung beim Getränkehändler ab.
Festausgabe	Eine einzelne Durchführung des Rattenfests. Sortiment, Bestände, Bestellungen und Rückgaben sind jeweils einer Festausgabe zugeordnet, damit die Daten mehrerer Jahre erhalten und vergleichbar bleiben.
Gebinde	Verkaufseinheit, in der ein Artikel geliefert und ausgegeben wird, zum Beispiel eine einzelne Flasche oder eine Schrumpfpackung. Bestellmengen müssen der Gebindegrösse entsprechen.
Getränkechef	Mitglied des RF-OK, das Festausgaben, Benutzer, Sortiment, Bars und Abrechnung verwaltet.
Lagerbestand	Die im Lager verfügbaren Mengen. Er setzt sich aus Artikeln des Sortiments mit der jeweiligen Anzahl zusammen. Zu Beginn des Fests entspricht er dem Anfangsbestand, danach verändert er sich durch Bezüge, Rückgaben und Korrekturen. Kurz auch Bestand.
Lagerteam	Mitarbeitende im zentralen Getränkelager. Sie nehmen während des Fests die Bestellungen entgegen, stellen sie bereit und übergeben sie den Bars.
RF-OK	Organisationskomitee des Rattenfests. Es wechselt jährlich.
Rückgabe	Rücknahme nicht verbrauchter Getränke einer Bar am Ende des Fests. Sie wird dem Bestand erst nach Bestätigung durch Lagerteam gutgeschrieben. Auch Retoure genannt.
Schrumpfpackung	Packung aus mehreren Getränken, die nur als Ganzes bezogen werden kann, zum Beispiel ein 6er-Pack.
Sortiment	Die Artikel, die für eine Festausgabe beim Getränkehändler bezogen werden können. Das Sortiment enthält keine Mengen.

Tabelle 4: Glossar

11 Hilfsmittelverzeichnis

Die folgende Übersicht dokumentiert die im Projekt verwendeten Hilfsmittel nach Aufgabenbereich. Die konkreten Tools werden im Verlauf des Projekts ergänzt.

Aufgabenbereich	Tools
Literatur-Recherche und Verwaltung	Google, DuckDuckGo, scholar.google.com
Datenanalyse und Visualisierung	
Ideengenerierung	Claude, Gemini, Typst
Übersetzung	
Qualitätsoptimierung des Textes (Synonyme/Formulierung von Begriffen und Satzstruktur)	GPT-5.6 Luna Anonymized
Prototyping	Figma
Coding	Claude Code
Texterstellung, Textoptimierung, Rechtschreibe- und Grammatikprüfung	Claude, Hunspell
Zusammenarbeit und Projektmanagement	Teams, GitHub, Outlook, Google Meet, Typst, WhatsApp
DevOps	GitHub
Notizen	Obsidian, Papier

Tabelle 5: Hilfsmittelverzeichnis

12 Bilderverzeichnis

Abbildung 1 Problem Domain der Getränkebestellung 8

Abbildung 2 Problem Domain der Getränkerückgabe am Ende des Festes 8

Abbildung 3 Bestellung durch eine Bar 9

Abbildung 4 Übersicht im Lager 9

Abbildung 5 Ansicht der Bar 9

Abbildung 6 Screenshot von REWE Webshop 13

Abbildung 7 Screenshot von just-eat.ch 14

Abbildung 8 Screenshot von jamezz.com/de 14

Abbildung 9 Stack Overflow Umfrage 2025 26

Abbildung 10 Beschreibung 26

Abbildung 11 Beschreibung 27

Abbildung 12 Langfristiger Zeitplan 33

13 Tabellenverzeichnis

Tabelle 1	9
Tabelle 2 Kategorien der MoSCoW-Klassifikation	21
Tabelle 3 Klassifikation der Functional Requirements	22
Tabelle 4 Glossar	34
Tabelle 5 Hilfsmittelverzeichnis	35

14 Bibliografie

- [1] shadcn, «shadcn/ui». Zugriffen: 27. September 2026. [Online]. Verfügbar unter: <https://ui.shadcn.com/>
- [2] «Event inventory application». [Online]. Verfügbar unter: <https://rapidstock.com/event-inventory-application>
- [3] S. Rogelberg, «Starbucks quietly retired its AI agent just months after deployment after it miscounted coffee shop inventories and slowed down baristas», *Fortune*, Mai 2026, [Online]. Verfügbar unter: <https://fortune.com/2026/05/28/starbucks-quietly-retired-ai-inventory-agent-barista-complaints-hallucinations>
- [4] «Accepting Contactless (NFC) Payments - SumUp - Support Center». [Online]. Verfügbar unter: <https://help.sumup.com/en-US/articles/722OhVEWYtfmy7X4959aJi-accepting-contactless-nfc-payments>
- [5] «QR-Code-Sticker - TWINT». [Online]. Verfügbar unter: <https://www.twint.ch/geschaefstkunden/produkte/twint-vor-ort/qr-code-sticker>
- [6] «Getraenke online bestellen und liefern lassen - REWE.de». [Online]. Verfügbar unter: <https://www.rewe.de/shop/c/getraenke-genussmittel/?source=mc>
- [7] K. Schmidt, «Smartphone-Apps der Bundesliga-Clubs: Ein Bier per Klick», *DER SPIEGEL*, Aug. 2011, [Online]. Verfügbar unter: <https://www.spiegel.de/sport/fussball/smartphone-apps-der-bundesliga-clubs-ein-bier-per-klick-a-779121.html>
- [8] R. Hillebrand, «Innovation im Stadion: Bier via App liefern lassen», *Salzburger Nachrichten*, Dez. 2025, [Online]. Verfügbar unter: <https://www.sn.at/panorama/medien/innovation-im-stadion-bier-via-app-liefern-lassen-art-164354>
- [9] F. Tepper, «VenueNext Raises 9M To Create Apps For Stadiums \vert TechCrunch», *TechCrunch*, Juni 2015, [Online]. Verfügbar unter: <https://techcrunch.com/2015/06/24/venue-next-raises-9m-to-create-apps-for-stadiums>
- [10] «Sports & Entertainment \vert Point of Sale & Payment Processing \vert Shift4». [Online]. Verfügbar unter: <https://www.shift4.com/sports-entertainment#contact>
- [11] «Sky Sport». [Online]. Verfügbar unter: <https://sport.sky.de/fussball/artikel/innovative-idee-stadionwurst-per-app-an-den-platz/11353472/33896>
- [12] «OrdiFy: Snacks und Getraenke im Stadion direkt an den Platz bestellen». [Online]. Verfügbar unter: <https://www.mobiflip.de/ordify-snacks-und-getraenke-im-stadion-direkt-an-den-platz-bestellen>
- [13] «Mit OrdiFy nie mehr am Stadion-Kiosk anstehen \vert Android User». [Online]. Verfügbar unter: <https://www.android-user.de/mit-ordify-nie-mehr-am-stadion-kiosk-anstehen>

- [14] «Borussia Dortmund revolutioniert Stadion-Erlebnis: Neue App-Funktion in Planung». [Online]. Verfügbar unter: <https://www.wa.de/nordrhein-westfalen/borussia-dortmund-revolutioniert-stadion-erlebnis-neue-app-funktion-in-planung-93586915.html>
- [15] pez/pd, «Just Eat und der FCSG starten Pickup-Service im Kybunpark», *stgallen24*, Feb. 2023, [Online]. Verfügbar unter: <https://stgallen24.ch/articles/176202-just-eat-und-der-fcsg-starten-pickup-service-im-kybunpark>
- [16] «Beyond the Game: Initiatives Powered by our UEFA Partnership». [Online]. Verfügbar unter: <https://newsroom.justeattakeaway.com/en-WW/251201-beyond-the-game-initiatives-powered-by-our-uefa-partnership>
- [17] Jamezz, «Jamezz». [Online]. Verfügbar unter: <https://jamezz.com/de/branchen/stadien>
- [18] Jamezz, «Jamezz», *Jamezz*, Apr. 2026, [Online]. Verfügbar unter: <https://jamezz.com/de/online-bestell-webshop>
- [19] «AWS End User Messaging Pricing». [Online]. Verfügbar unter: <https://aws.amazon.com/de/end-user-messaging/pricing>
- [20] L.-M. Bogza u. a., «User-centered design and evaluation of a web-based decision aid for older adults living with mild cognitive impairment and their health care providers: mixed methods study», *Journal of medical Internet research*, Bd. 22, Nr. 8, S. e17406, 2020.
- [21] W3c, «Use Clear and Understandable Content», *Web Accessibility Initiative (WAI)*, Sep. 2026, [Online]. Verfügbar unter: <https://www.w3.org/WAI/WCAG2/supplemental/objectives/o3-clear-content>
- [22] W3c, «Home», *Web Accessibility Initiative (WAI)*, Sep. 2026, [Online]. Verfügbar unter: <https://www.w3.org/WAI>
- [23] W3c, «WCAG 2 Overview», *Web Accessibility Initiative (WAI)*, Juni 2018, [Online]. Verfügbar unter: <https://www.w3.org/WAI/standards-guidelines/wcag>
- [24] «How to Meet WCAG (Quickref Reference)». [Online]. Verfügbar unter: <https://www.w3.org/WAI/WCAG22/quickref>
- [25] F. J. Pathari, Y. Nielsen, L. I. Andersen, und G. Marentakis, «Dark vs. light mode on smartphones: Effects on eye fatigue», in *Proceedings of the ACHI 2024 The Seventeenth International Conference on Advances in Computer-Human Interactions*, 2024, S. 26–30.
- [26] J. Laine, «Impact of dark mode on user experience and eye comfort», 2025.
- [27] «Angular vs React: 1 Clear Winner in 2026 [7 Tests]». [Online]. Verfügbar unter: <https://tech-insider.org/angular-vs-react-2026>
- [28] «Technology \vert 2025 Stack Overflow Developer Survey». [Online]. Verfügbar unter: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-webframe-prof>
- [29] «Create T3 App». [Online]. Verfügbar unter: <https://create.t3.gg/>
- [30] «How to Choose the Right Front-End Framework for Your Project». [Online]. Verfügbar unter: <https://devtechinsights.com/choose-right-frontend-framework>
- [31] «React Versions – React». [Online]. Verfügbar unter: <https://react.dev/versions>
- [32] «Versioning Policy – React». [Online]. Verfügbar unter: <https://react.dev/community/versioning-policy>

- [33] A. Team, «Installation \ifmmode \bullet \else •\fi Angular». [Online]. Verfügbar unter: <https://angular.dev/installation>
- [34] A. Team, «Versioning and releases \ifmmode \bullet \else •\fi Angular». [Online]. Verfügbar unter: <https://angular.dev/reference/releases>
- [35] A. Team, «Push notifications \ifmmode \bullet \else •\fi Angular». [Online]. Verfügbar unter: <https://angular.dev/ecosystem/service-workers/push-notifications>
- [36] S. Khati Chhetri, «Comparative Study of Front-end Frameworks : React and Angular». [Online]. Verfügbar unter: <https://www.theseus.fi/items/264baf5d-1b0e-4489-9fcb-93bd65d16223>
- [37] «Vue.js». [Online]. Verfügbar unter: <https://vuejs.org/>
- [38] G. Sourabh, «Vue vs React: Which is the Best Frontend Framework in 2025? \vert BrowserStack», *BrowserStack*, Aug. 2025, [Online]. Verfügbar unter: <https://www.browserstack.com/guide/react-vs-vuejs>
- [39] guardrex, «ASP.NET Core Blazor». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-10.0#build-a-full-stack-web-app-with-blazor>
- [40] «Blazor vs React Performance - Which Is Faster in 2025?». [Online]. Verfügbar unter: <https://brisktechsol.com/blazor-vs-react>
- [41] «Blazor tutorial - Build your first web app \vert .NET». [Online]. Verfügbar unter: <https://dotnet.microsoft.com/en-us/learn/aspnet/blazor-tutorial/intro>
- [42] guardrex, «ASP.NET Core Blazor Progressive Web Application (PWA)». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/aspnet/core/blazor/progressive-web-app/?view=aspnetcore-10.0&tabs=visual-studio>
- [43] danroth27, «Build reusable UI components with Blazor - .NET». [Online]. Verfügbar unter: <https://learn.microsoft.com/en-us/dotnet/architecture/blazor-for-web-forms-developers/components>

15 Anhang A

15.1 APIs

15.2 Screenshots

15.3 Testreports

16 Anhang B

16.1 Eigenständigkeitserklärung